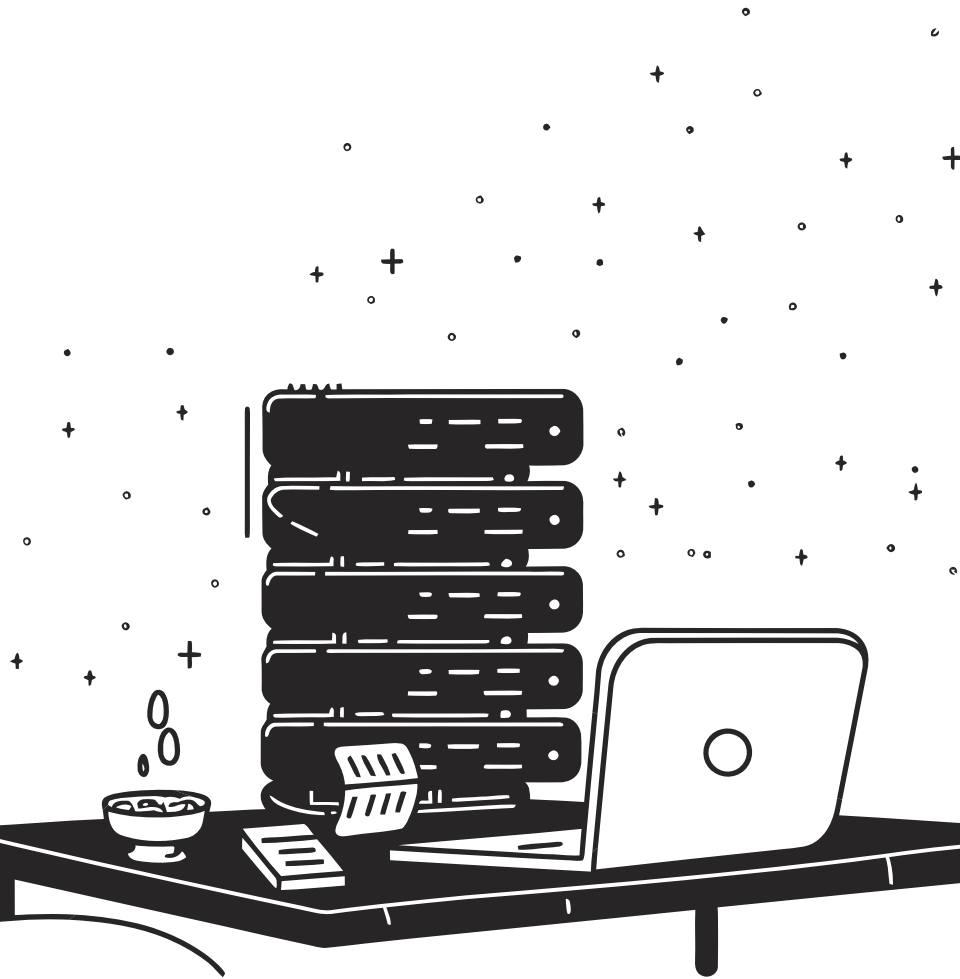




Teoria R2 — POO, Composer i proves unitàries

Introducció pràctica per a R2M8: com encapsular una regla de negoci en una classe simple, gestionar càrregues amb Composer i verificar el comportament amb proves automàtiques.

CICLE FORMATIU · DAW / DAM

Què farem en aquest R2?

El mínim imprescindible per introduir POO sense reescriure tot el projecte.

01

Regla de domini

Extraure una regla procedural a una classe simple amb un mètode pur.

03

Prova unitària

Un test mínim que comprova dos casos sense obrir cap navegador.

02

Composer i autoload

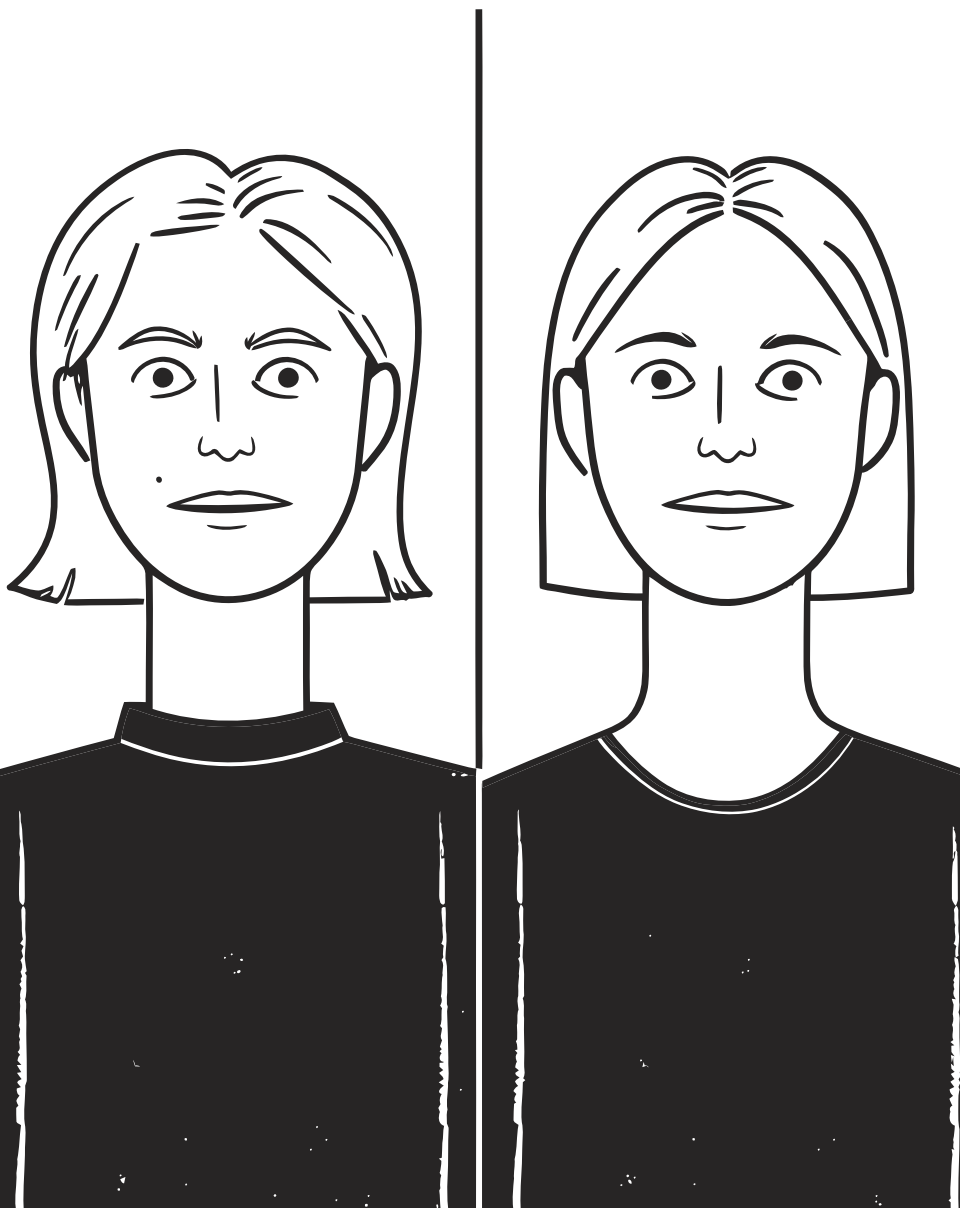
Definir PSR-4 al `composer.json` i eliminar les càrregues manuals disperses.

04

No regressió del flux

Demostrar que l'aplicació web continua funcionant exactament igual que abans.

 MVC complet, ORM i frameworks queden per a etapes posteriors. Ara, el fonament.



De procedural a classe: per què?

Abans de fer el canvi, convé entendre on viveix ara la regla i quins problemes provoca.

● Regla com a codi procedural

```
// Dins de processa_reserva.php
if ($estat === 'pendent' &&
    !$teConflicte) {
    // confirma la reserva
}
```

La regla queda **enterrada** dins del flux web, barrejada amb POST, sessió i HTML. Difícil de reutilitzar i impossible de provar sola.

● Regla encapsulada en una classe

```
$regla = new App\ReglaReserva();
if ($regla->potConfirmar($estat,
    $teConflicte)) {
    // confirma la reserva
}
```

La lògica viu en un lloc únic, té nom propi i es pot provar de forma **aïllada**, sense navegador ni base de dades.

La classe ReglaReserva

Una classe simple amb un únic mètode pur: rep dades, retorna un resultat. Sense dependències globals.

```
namespace App;

final class ReglaReserva
{
    public function potConfirmar(string $estat, bool $teConflicte): bool
    {
        return $estat === 'pendent' && !$teConflicte;
    }
}
```

final

Evita herències no previstes.

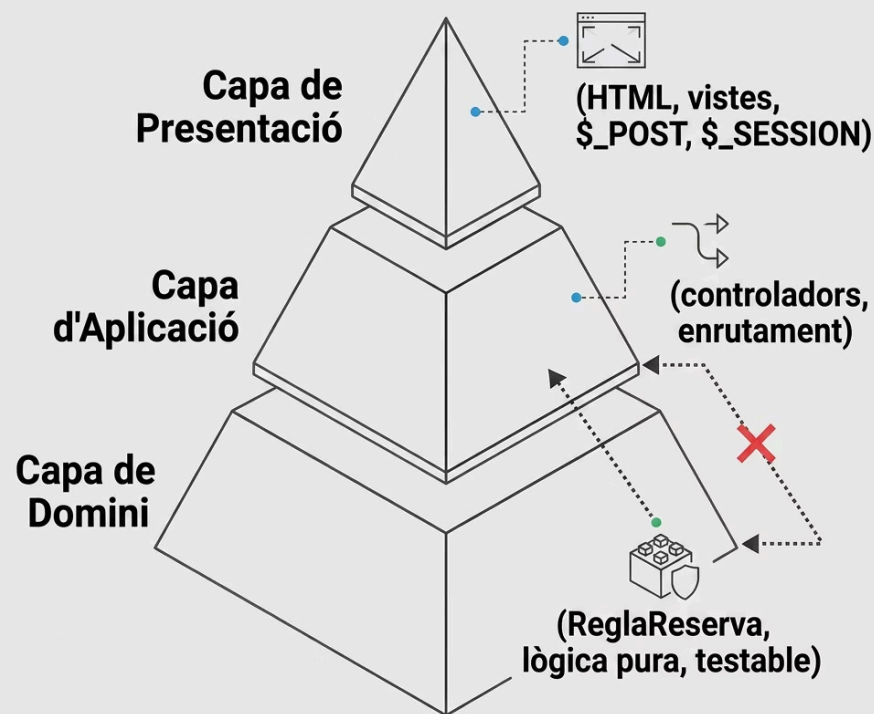
Tipus declarats

string i bool eviten errors silenciosos.

Sense globals

No llegeix \$_POST, \$_SESSION ni genera HTML.

Per què la classe no ha de dependre de `$_POST` ni HTML?



Principi de separació de responsabilitats

Una classe de domini encapsula **lògica de negoci**. Si llegeix directament `$_POST` o emet HTML, queda acoblada a la capa de presentació.

- No es pot provar sense simular una petició HTTP.
- No es pot reutilitzar en una API, CLI o altre context.
- Un canvi al formulari pot trencar la lògica de negoci.

✅ La classe rep **paràmetres**, no l'entorn. Qui crida la classe s'encarrega d'extreure els valors de POST o sessió.



Composer i autoload PSR-4

Composer elimina els `require_once` dispersos i carrega automàticament qualsevol classe del projecte.

composer.json

```
{
  "autoload": {
    "psr-4": {
      "App\\": "src/"
    }
  }
}
```

Després d'editar el fitxer, executa:

```
composer dump-autoload
```

Ús a l'aplicació

```
require_once __DIR__ .
'../vendor/autoload.php';

// A partir d'aquí, totes les classes
// de src/ es carreguen
automàticament
$regla = new App\ReglaReserva();
```

i PSR-4 mapeja l'espai de noms `App\` amb el directori `src/`.
`App\ReglaReserva` correspon a `src/ReglaReserva.php`.

La prova unitària mínima

Un test comprova una peça xicoteta de forma aïllada, sense navegador ni base de dades. Dos casos bàsics: el cas que ha de passar i el que no.

```
require_once __DIR__ . '/../vendor/autoload.php';

$regla = new App\ReglaReserva();

// Cas 1: pendent sense conflicte → ha de confirmar
assert($regla->potConfirmar('pendent', false) === true,
    'Error: hauria de poder confirmar');

// Cas 2: pendent amb conflicte → no ha de confirmar
assert($regla->potConfirmar('pendent', true) === false,
    'Error: no hauria de confirmar amb conflicte');
```

Cas feliç ✓

Comprova que la regla aprova quan totes les condicions es compleixen.

Cas negatiu ✗

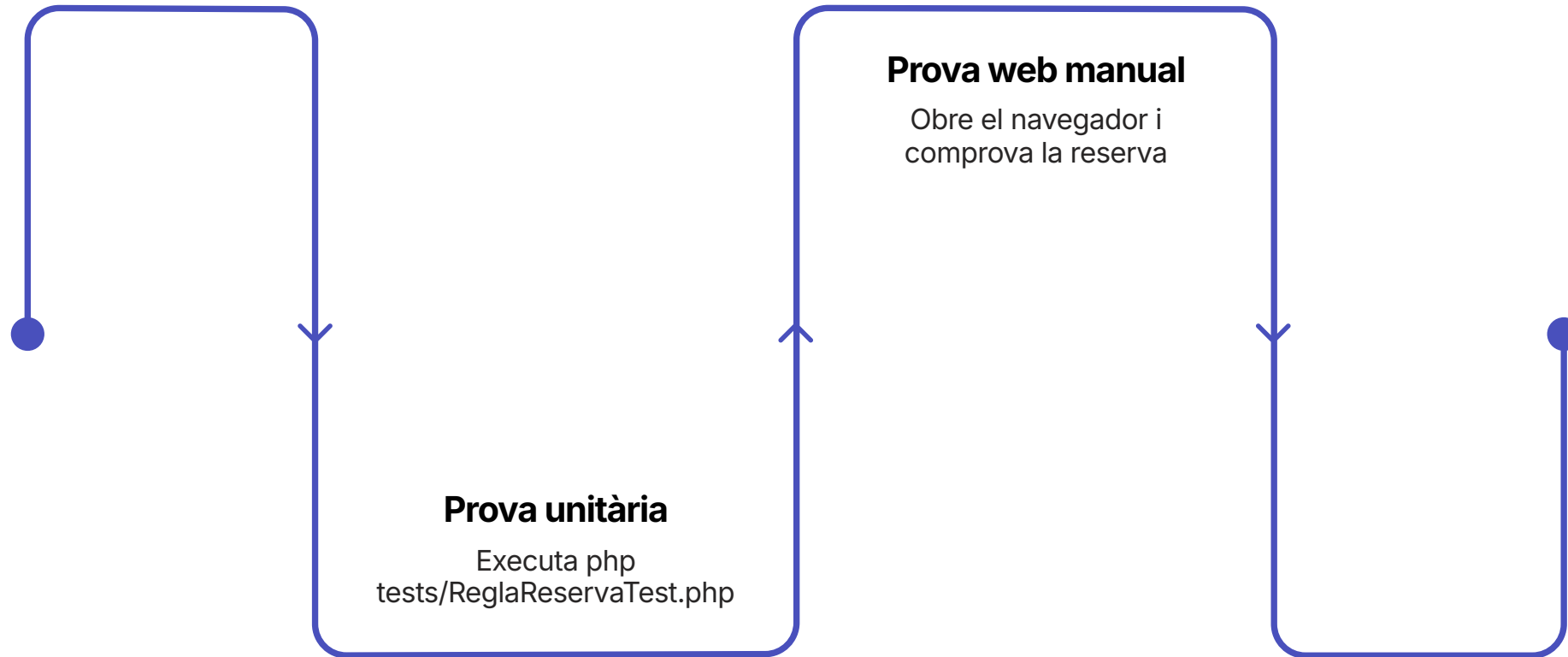
Comprova que la regla rebutja quan hi ha un conflicte, evitant falsos positius.

Missatge clar 💬

Cada assert porta un missatge per saber exactament quin cas ha fallat.

No regressió del flux web

Extraure una classe **no ha de trencar l'aplicació**. Cal demostrar-ho explícitament.



La prova del flux comprova que la petició HTTP, els paràmetres de POST i la resposta HTML continuen funcionant correctament després del refactoring. Són dues verificacions complementàries, no alternatives.



Errors habituals

Classe que llegeix `$_POST` directament

Acobla la lògica a la capa web. La solució és rebre els valors com a paràmetres del mètode.

Oblidar `composer dump-autoload`

Després de crear una classe nova o modificar el `composer.json`, cal regenerar el mapa d'autoload o PHP no trobarà la classe.

Namespace incorrecte o fitxer mal ubicat

`App\ReglaReserva` ha d'estar a `src/ReglaReserva.php`. Un error de majúscules o de carpeta trenca l'autoload.

Provar només el camí feliç

Si no hi ha un cas negatiu, el test no protegeix contra errors de lògica invertida o condicions incorrectes.

Checklist de lliurament R2

Comprova cada punt abans de fer el lliurament. Tot ha d'estar en verd.

Codi

- ✓ ~~Una classe al directori `src/` amb namespace `App\`~~
- ✓ ~~El mètode no llegeix `$_POST`, `$_SESSION` ni emet HTML~~
- ✓ ~~`composer.json` amb autoload PSR-4 configurat~~
- ✓ ~~`vendor/autoload.php` inclòs al punt d'entrada de l'app~~
- ✓ ~~Fitxer de test amb almenys dos casos (`assert` o equivalent)~~

Preguntes de reflexió

- Quina regla has extret a una classe simple?
- Per què la classe no depèn de POST ni HTML?
- Com demostres que el flux web continua funcionant?

- ✓ Si pots respondre les tres preguntes i el checklist és verd, estàs llest per entregar R2.

Preguntes de comprovació

Per assegurar la comprensió dels conceptes clau, revisa les següents preguntes:

? Quina regla del flux has tret a una classe simple?

? Per què la classe no ha de dependre directament de `$_POST` o HTML?

? Què aporta Composer/autoload en aquest punt?

? Quin test mínim comprova la classe?

? Com demostres que el flux web continua funcionant després del canvi?